UNIT III

Generative Models in NLP

Topics Covered

Transformer Architecture Recap (BERT, GPT), GPT-3/4, PaLM, Claude, and LLaMA Architectures, Text Generation Pipelines and APIs (OpenAI, HuggingFace), Prompt Engineering with GPT Models, Fine-tuning vs Instruct Tuning, Retrieval-Augmented Generation (RAG), Evaluation Metrics: BLEU, ROUGE, Perplexity, Building LLM-based Apps with LangChain

3.1. Transformer Architecture Recap — BERT & GPT

Foundation: Transformer Architecture

Every model in this unit is built on the Transformer, introduced in **'Attention Is All You Need**' (Vaswani et al., 2017). Before diving into GPT-4, Claude, and LLaMA, it's essential to recap the two founding Transformer families — BERT and GPT — since almost every modern LLM design choice traces back to one or both.

Core Definition

A Transformer is a neural network architecture built entirely on self-attention, allowing every token in a sequence to directly attend to every other token, rather than processing tokens strictly in order as older RNN/LSTM architectures did.

The original Transformer is an encoder–decoder architecture. It contains two major components:

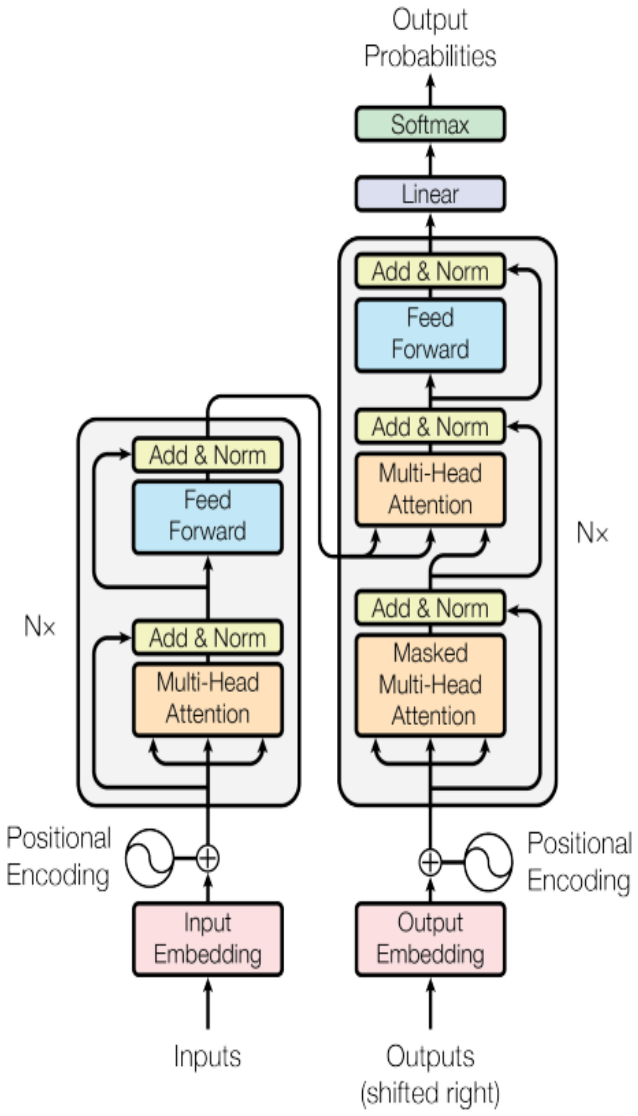
- Encoder: Understands the input sequence and creates contextual representations.
- Decoder: Generates the output sequence one token at a time, using the encoded input and previously generated tokens.

The diagram shows the **Encoder–Decoder Transformer architecture**. It converts an input sequence into an output sequence using **attention mechanisms**.

Encoder — Understands the Input (The left block labeled N× **is the** Encoder**)**

1. **Input Embedding** – Converts input words/tokens into numerical vectors.
2. **Positional Encoding** – Adds information about the position/order of each token.
3. **Multi-Head Self-Attention** – Allows each token to focus on other relevant tokens in the input.
4. **Add & Norm** – Uses a residual connection and layer normalization to stabilize learning.

5. **Feed Forward Network** – Further transforms each token representation using neural-network layers.
6. **N ×** – The encoder layer is repeated multiple times; the original Transformer uses **6 layers**.



Decoder — Generates the Output (The right block labeled N× **is the** Decoder**)**

1. **Output Embedding** – Converts previously generated output tokens into vectors.
2. **Positional Encoding** – Adds position information to output tokens.
3. **Masked Multi-Head Attention** – Looks only at previous output tokens, preventing the decoder from seeing future tokens.
4. **Add & Norm** – Residual connection + normalization.

5. **Multi-Head Attention (Cross-Attention)** – Connects the decoder to the **encoder output** and identifies relevant input information.
6. **Add & Norm** – Again applies residual connection and normalization.
7. **Feed Forward Network** – Processes the resulting representations.
8. **N ×** – The decoder layer is also repeated multiple times.

Output Generation

1. **Linear Layer** – Converts the decoder representation into scores for all possible vocabulary tokens.
2. **Softmax** – Converts these scores into probabilities.
3. **Output Probabilities** – The token with the appropriate/highest probability is selected as the next output token.

Transformer Architecture — English to Telugu Example

Let us understand the Transformer **step by step** using:

Input: "I love India."

Output: "నేను భారతదేశాన్ని ప్రేమిస్తున్నాను."

Step 1: Input Sentence

The user gives the English sentence:

"I love India."

The Transformer receives this sentence as the input sequence.

Step 2: Tokenization

The sentence is divided into smaller units called **tokens**:

I | love | India

These tokens are converted into numerical IDs that the model can process.

Step 3: Word Embeddings

Each token is converted into a numerical vector called an **embedding**.

The embedding represents the semantic information of each token.

For example, the model learns that **"love"** is related to an action or emotion, while **"India"** represents a country.

Step 4: Positional Encoding

The Transformer needs to know the position of each token.

Therefore, positional information is added:

- **I** → first position
- **love** → second position
- **India** → third position

This helps the model understand the order of the sentence.

Step 5: Encoder Processes the Input

The Encoder receives the embedded and position-enhanced tokens.

Its main purpose is to **understand the meaning and context of the English sentence**.

Step 6: Self-Attention

The Encoder uses **Self-Attention** to understand relationships between the words.

For example, the model learns that:

- **"I"** is associated with the person performing the action.
- **"love"** represents the action.
- **"India"** is associated with the object of the action.

Thus, the Encoder develops a contextual understanding of:

"I love India."

Step 7: Encoder Creates Contextual Representation

After passing through the Encoder layers, the sentence is represented as contextual numerical information.

This representation contains information about the relationships and meaning of the English sentence.

Step 8: Decoder Starts Generating Telugu

The Decoder now begins generating the Telugu translation.

It starts with a special token such as **<START>**.

The first predicted token is:

"నేను"

Step 9: Decoder Uses Masked Self-Attention

When generating the next token, the Decoder can look at the tokens it has already generated.

After generating:

"నేను"

it uses this information to predict what should come next.

It cannot look at future Telugu tokens that have not yet been generated.

Step 10: Cross-Attention

The Decoder also looks at the Encoder's representation of:

"I love India."

This is called **Cross-Attention**.

It helps the Decoder identify which parts of the English input are relevant while generating each Telugu token.

Step 11: Generate the Next Tokens

The Decoder progressively generates:

నేను

then:

నేను భారతదేశాన్ని

then:

నేను భారతదేశాన్ని ప్రేమిస్తున్నాను

More precisely, for the intended Telugu translation, it generates:

నేను భారతదేశాన్ని ప్రేమిస్తున్నాను

The exact tokenization may split Telugu words into multiple subword tokens internally.

Step 12: Prediction Using Softmax

For every generation step, the Decoder produces probabilities for possible next tokens.

The token with the selected highest-probability prediction is generated, and the process continues.

Step 13: End of Sentence

When the Decoder generates an **<END>** token, the translation is complete.

The final output is:

"నేను భారతదేశాన్ని ప్రేమిస్తున్నాను."

Transformer Architecture Recap — BERT

BERT (Bidirectional Encoder Representations from Transformers) is a Transformer-based language model introduced by Google in 2018.

The central idea behind BERT is **contextual understanding**. Instead of understanding a word independently, BERT determines the meaning of a word by considering the **words surrounding it from both directions**.

For example:

"The bank is near the river."

Here, the word **bank** is understood in the context of **river**.

If the sentence is:

"The bank approved my loan."

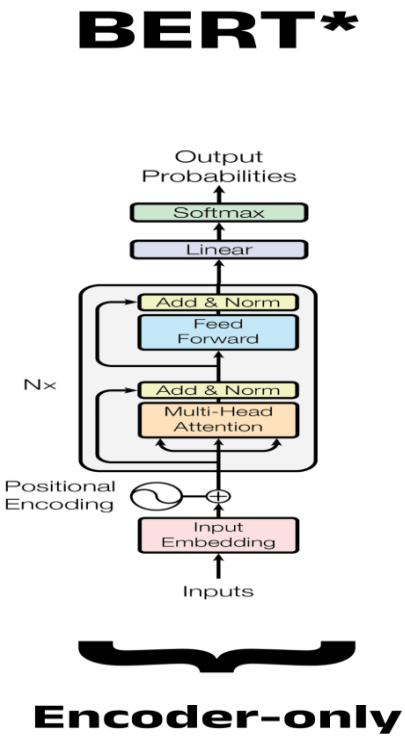
the surrounding context changes, so the representation of **bank** also changes.

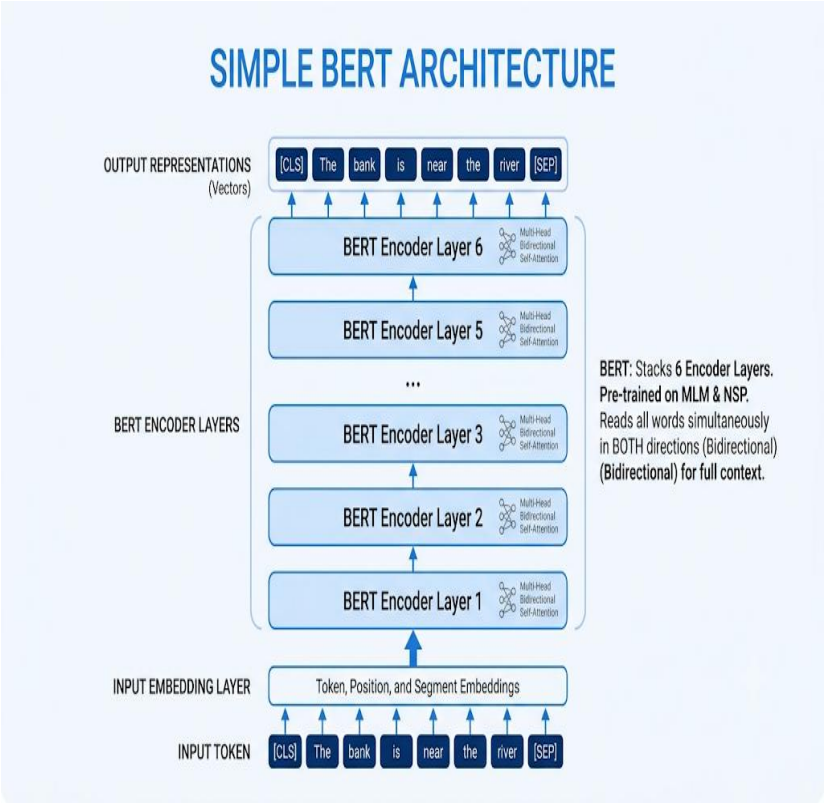
This ability comes from the **Self-Attention mechanism** of the Transformer Encoder.

Core idea

BERT = Transformer Encoder + Bidirectional Contextual Understanding

BERT is primarily designed for **understanding and representing language**, rather than generating text like GPT.





Step by Step Explanation for BERT Architecture

Step 1: Input to BERT

BERT receives text as input.

For example:

"The bank is near the river."

The text is converted into tokens:

[CLS] The bank is near the river . [SEP]

Two special tokens are important:

[CLS] — placed at the beginning and used for sequence-level tasks such as classification.

[SEP] — identifies the end of a sentence or separates two sentences.

Step 2: Input Representation

BERT creates the representation of each token using three components:

Token Embedding + Segment Embedding + Position Embedding ----------> BERT Input Representation

Token Embedding: Represents the token itself.

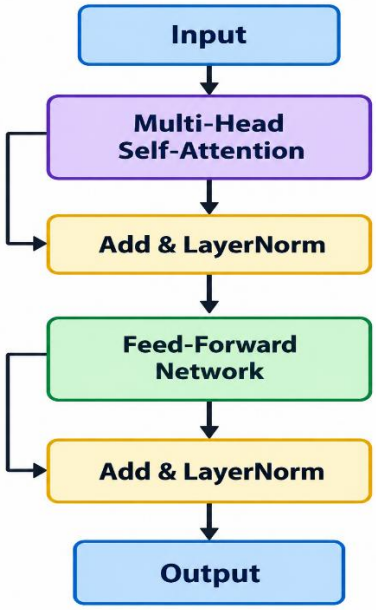
Position Embedding: Provides information about the token's position in the sequence.

Segment Embedding: Indicates which sentence a token belongs to when BERT receives a pair of sentences.

Step 3: Transformer Encoder

The input representation enters the Transformer Encoder.

Each Encoder layer contains:



This Encoder structure is repeated many times.

For example:

- **BERT-Base → 12 Encoder layers**
- **BERT-Large → 24 Encoder layers**

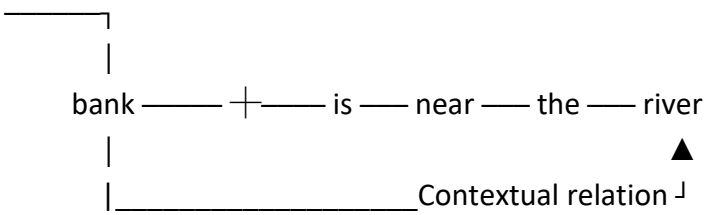
Step 4: Multi-Head Self-Attention

This is the key mechanism that allows BERT to understand relationships between words.

Consider:

"The bank is near the river."

When processing **bank**, BERT can relate it to:



The model calculates how strongly different tokens should contribute to each token's representation.

With **multiple attention heads**, different relationships can be captured simultaneously.

Step 5: Feed-Forward Network

After self-attention, each token representation passes through a feed-forward neural network.

Step 6: Contextual Representation

After passing through all Encoder layers, BERT produces a contextual representation for every token.

For example:

The → Contextual Vector

bank → Contextual Vector

is → Contextual Vector

near → Contextual Vector

river → Contextual Vector

The important point is that the representation of **bank** depends on its context.

Compare:

"river bank"

with

"bank approved my loan"

The word is the same, but its contextual representation is different.

Step 7: BERT Pretraining

BERT was originally pretrained using two objectives.

Masked Language Modeling — MLM

A portion of the input is masked:

The bank is near the [MASK].

BERT predicts: river

This trains BERT to understand the relationship between surrounding words.

Next Sentence Prediction — NSP

BERT's original training also included a task where it learned whether one sentence followed another.

Step 8: BERT Fine-Tuning

BERT Fine-Tuning is a transfer learning strategy where a general, pre-trained BERT model (which already understands grammar, syntax, and word meanings) is further trained on a smaller, task-specific labeled dataset. Instead of training a model from scratch, you add a lightweight output layer (classifier head) to BERT and train the entire network end-to-end using a low learning rate.

The basic idea is:

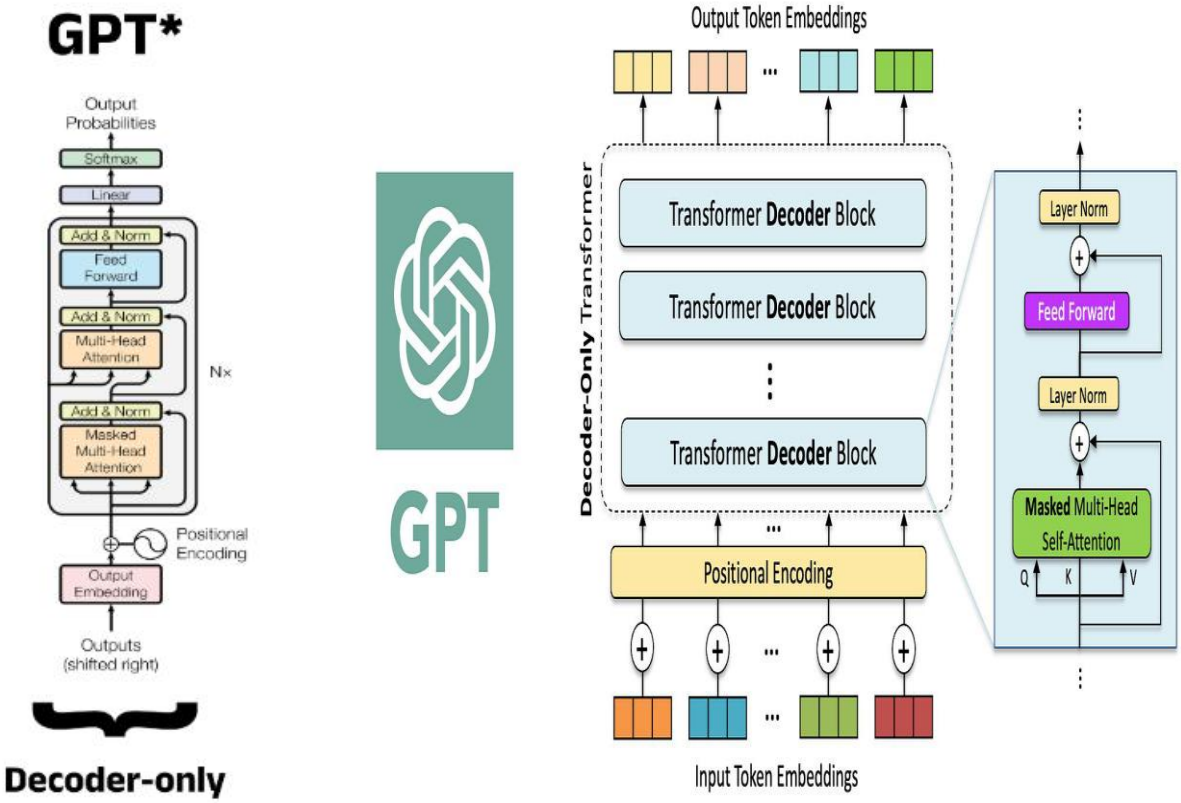
Pretrained BERT + Task-specific data + Training → Fine-tuned BERT

Primary Uses & Applications

- **Text Classification & Sentiment Analysis:** Categorizing emails as Spam/Ham, customer reviews as Positive/Negative, or news articles into specific categories.
- **Named Entity Recognition (NER):** Extracting specific entities from unstructured text—such as identifying names, locations, organizations, dates, or medical terms.
- **Question Answering Systems:** Powering search engines or customer support bots to extract precise answer spans directly from reference passages (e.g., SQuAD benchmark tasks).
- **Semantic Textual Similarity & NLI:** Detecting duplicate questions (e.g., Quora dataset) or evaluating logical relationships between sentence pairs (Entailment, Contradiction, Neutral).

Transformer Architecture Recap — GPT

Generative Pre-trained Transformer (**GPT**) models are built on a **decoder-only variant** of the original Transformer architecture introduced in the 2017 paper *"Attention Is All You Need"*. Unlike encoder-decoder transformers (used for translation) or encoder-only models like BERT (used for comprehension), GPT is designed strictly for **autoregressive text generation**—predicting the next token given all previous tokens.



GPT Components

1. Tokenization

The input sentence is divided into tokens. For example:

"AI is powerful" → AI | is | powerful

2. Token Embedding

Each token is converted into a numerical vector that represents its learned semantic information.

3. Positional Information

Position information is incorporated so that the model understands the order of tokens in the sequence.

4. Masked Self-Attention

GPT uses **causal/masked self-attention**. A token can attend only to itself and previous tokens, not future tokens.

T1 → T1

T2 → T1,T2

T3 → T1,T2,T3

T4 → T1,T2,T3,T4

5. Feed-Forward Network

After attention, each token representation passes through a fully connected neural network to perform further nonlinear transformation.

6. Residual Connections and Layer Normalization

These improve information flow and help stabilize the training of deep Transformer networks.

7. Multiple Transformer Blocks

GPT stacks many identical Transformer blocks. As the information passes through the layers, increasingly sophisticated language representations are learned.

8. Linear Layer and Softmax

The final representation is converted into vocabulary scores (**logits**). Softmax converts these scores into probabilities for possible next tokens.

9. Next-Token Prediction

GPT predicts the most probable next token based on the preceding context.

Example

Step 1: Input: "The"

Predicted next token: "Student"

Step 2: Input: "The Student"

Predicted next token: "is"

Step 3: Input: "The Student is"

Predicted next token: "studying"

Step 4: Input: "The Student is studying"

Predicted next token: "a"

Step 5: Input: "The Student is studying a"

Predicted next token: "book"

Step 6: Input: "The Student is studying a book"

Predicted next token: "."

Final Generated Sentence

"The Student is studying a book."

At every step, GPT uses the previous tokens as context and predicts one next token.

3.2. GPT-3/4 Architecture

GPT-3 Architecture

GPT-3 is a decoder-only Transformer language model introduced by OpenAI in 2020. It uses an autoregressive architecture, meaning it predicts the next token based on the preceding tokens.

GPT-3 begins with a text prompt, which is converted into tokens using a subword tokenizer. Each token is mapped to a learned embedding vector. Learned positional embeddings are added to represent the order of tokens.

The resulting representations pass through a stack of Transformer decoder blocks. GPT-3's largest model, GPT-3 175B, contains 96 Transformer layers, 12,288-dimensional hidden representations, and 96 attention heads.

Each Transformer block contains:

1. Masked multi-head self-attention: Allows each token to attend to previous tokens while preventing access to future tokens.
2. Feed-forward neural network: Applies nonlinear transformations to learn complex patterns.
3. Residual connections and layer normalization: Support stable training and information flow through the deep network.

After passing through all Transformer layers, the final hidden representations are projected into the vocabulary space. A softmax function produces probabilities for the next token. The selected token is appended to the sequence, and the process repeats until generation is complete.

GPT-4 Architecture

GPT-4 is a more advanced GPT model introduced by OpenAI in 2023. It retains the autoregressive Transformer-based approach but introduces significant improvements in training, capability, and supported inputs.GPT-4 receives text input, and supported versions can also process images. The input is converted into internal representations and processed by a Transformer-based neural network.

The model uses causal attention to understand the relationships between tokens and predict the next token. Its internal layers transform these representations into increasingly complex contextual

information.The final representation is converted into vocabulary logits, and a probability distribution is generated. The model selects a token and continues generating the response autoregressively.

GPT-4 was trained using large-scale pre-training followed by alignment methods, including reinforcement learning from human feedback (RLHF). The exact architecture, number of parameters, number of layers, and attention heads were not disclosed in OpenAI's GPT-4 technical report.

Feature	GPT-1	GPT-3	GPT-4
Full name	Generative Pre-trained Transformer 1	Generative Pre-trained Transformer 3	Generative Pre-trained Transformer 4
Year introduced	2018	2020	2023
Architecture	Decoder-only Transformer	Decoder-only Transformer	Transformer-based autoregressive model
Parameters	117 million	175 billion (largest model)	Not publicly disclosed
Transformer layers	12	96 (largest model)	Not publicly disclosed
Attention heads	12	96 (largest model)	Not publicly disclosed
Original context length	512 tokens	2,048 tokens	8,192 tokens; GPT-4-32K supports 32,768
Input modality	Text	Text	Text and images in supported versions
Training objective	Next-token prediction	Next-token prediction	Next-token prediction
Training approach	Pre-training and task-specific fine-tuning	Large-scale pre-training and few-shot learning	Pre-training and alignment, including RLHF
Language generation	Basic generative capabilities	More advanced text generation	Improved text generation and instruction following
Reasoning capability	Limited compared with later models	Improved compared with GPT-1	Improved reasoning capabilities
Main contribution	Established the GPT architecture	Demonstrated the benefits of large-scale language modeling	Advanced GPT capabilities and multimodal input

3.3. PaLM (Pathways Language Model)

PaLM (Pathways Language Model) is a large-scale language model developed by Google and introduced in 2022. It is based on the **decoder-only Transformer architecture** and is designed for large-scale language understanding, reasoning, and generation.

- PaLM begins with **input text**, which is divided into smaller units called **tokens** using subword tokenization. These tokens are converted into numerical vectors through **token embeddings**. Positional information is also incorporated so that the model can understand the order of tokens in the input sequence.
- The token representations are then passed through a **stack of Transformer decoder layers**. These layers form the core of the PaLM architecture.
- Each Transformer layer mainly consists of **self-attention, feed-forward networks, residual connections, and layer normalization**. PaLM uses **causal self-attention**, which means that while predicting a particular token, the model can attend only to the preceding tokens and cannot look at future tokens.
- The **self-attention mechanism** enables PaLM to identify relationships between different tokens in the input. For example, in a sentence such as *"The student submitted the assignment because it was due today,"* attention helps the model understand the relationship between *student*, *assignment*, and *it*.
- The **feed-forward network** further transforms the representations produced by the attention mechanism. It introduces nonlinear processing and enables the model to learn complex linguistic and semantic patterns.
- After passing through the Transformer layers, the final representation is processed by an **output layer** that produces probability scores for all possible tokens in the vocabulary. The model selects or samples the next token based on these probabilities.

Autoregressive Generation

PaLM generates text **one token at a time**.

For example:

Input: "The capital of Andhra Pradesh is"

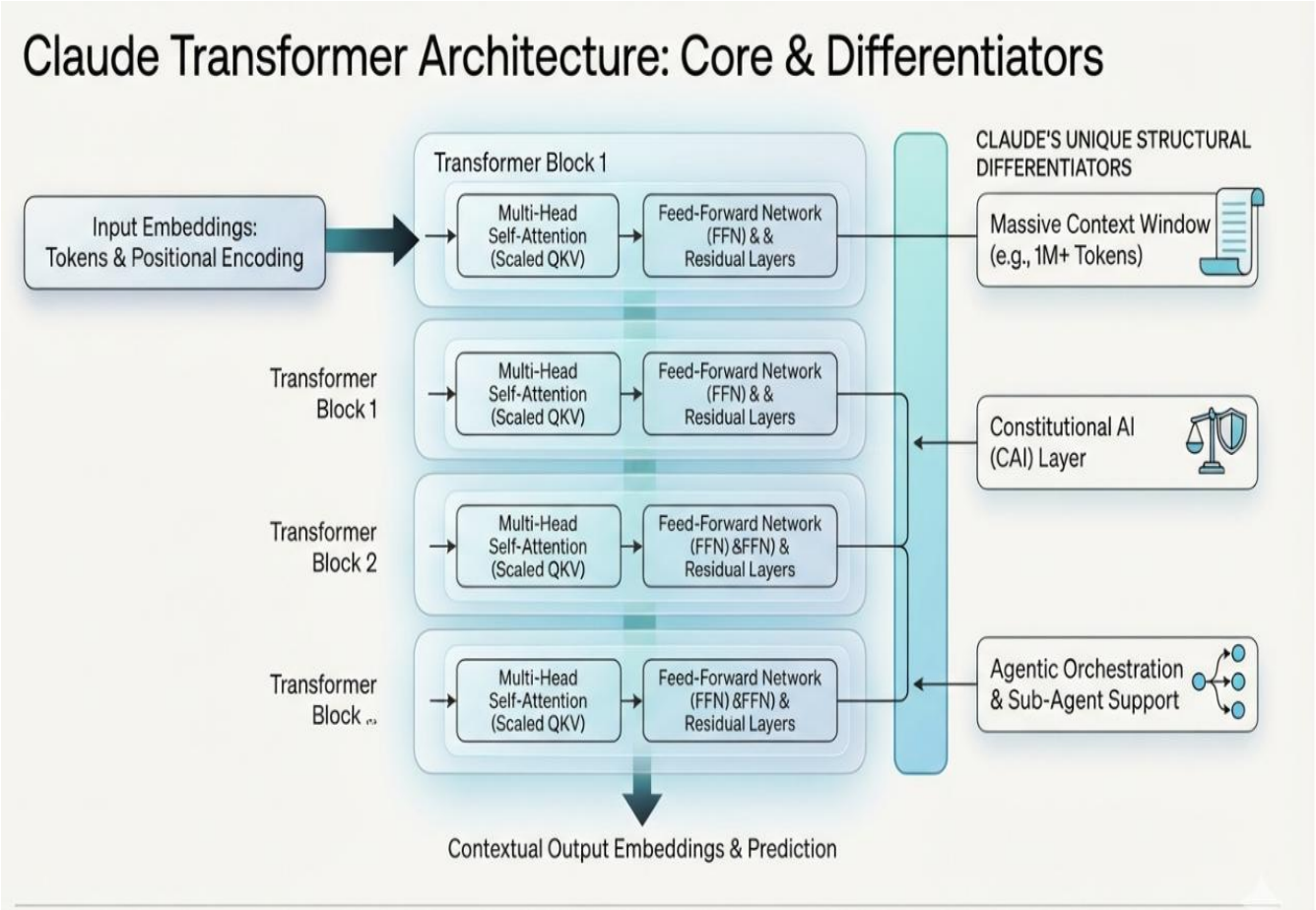
PaLM predicts: "Amaravati"

After predicting **Amravati**, the generated token becomes part of the context for predicting the next token. This process continues until the required response is generated.

3.4. Claude Architecure

Claude is a family of Large Language Models (LLMs) developed by **Anthropic**. At a high level, Claude uses a **Transformer-based neural network** for understanding and generating natural language. Its training and alignment also involve **Constitutional AI**, which is designed to guide the model toward useful, safe, and responsible responses.

Note: Anthropic does not publicly disclose all low-level architectural details of current Claude models, such as the exact number of layers, hidden dimensions, or attention heads.



Step-by-Step Architecture

Step 1: User Input

The user provides a natural-language prompt such as:

"Explain Artificial Intelligence."

Step 2: Tokenization

The input text is divided into smaller units called **tokens**. These tokens are converted into numerical identifiers that the model can process.

Step 3: Token Embedding

Each token is converted into a high-dimensional **vector representation**. These vectors capture learned linguistic information.

Step 4: Context and Position Information

The model processes the tokens in their sequence so that relationships and contextual information can be understood.

Step 5: Transformer Processing

The token representations pass through multiple **Transformer layers**. Each layer progressively builds a richer contextual representation.

Step 6: Self-Attention

Self-attention determines which tokens are important in relation to other tokens. It enables Claude to understand relationships between words across the input context.

Step 7: Feed-Forward Networks

After attention processing, feed-forward neural networks further transform the representations and learn complex patterns.

Step 8: Repeated Layer Processing

The combination of attention and feed-forward processing is repeated across the Transformer network, producing increasingly contextual representations.

Step 9: Next-Token Prediction

Claude predicts the next appropriate token based on the complete context available to it.

For example:

"The capital of India is" → "New Delhi"

The generated token is then included in the context for subsequent predictions.

Step 10. Response Generation

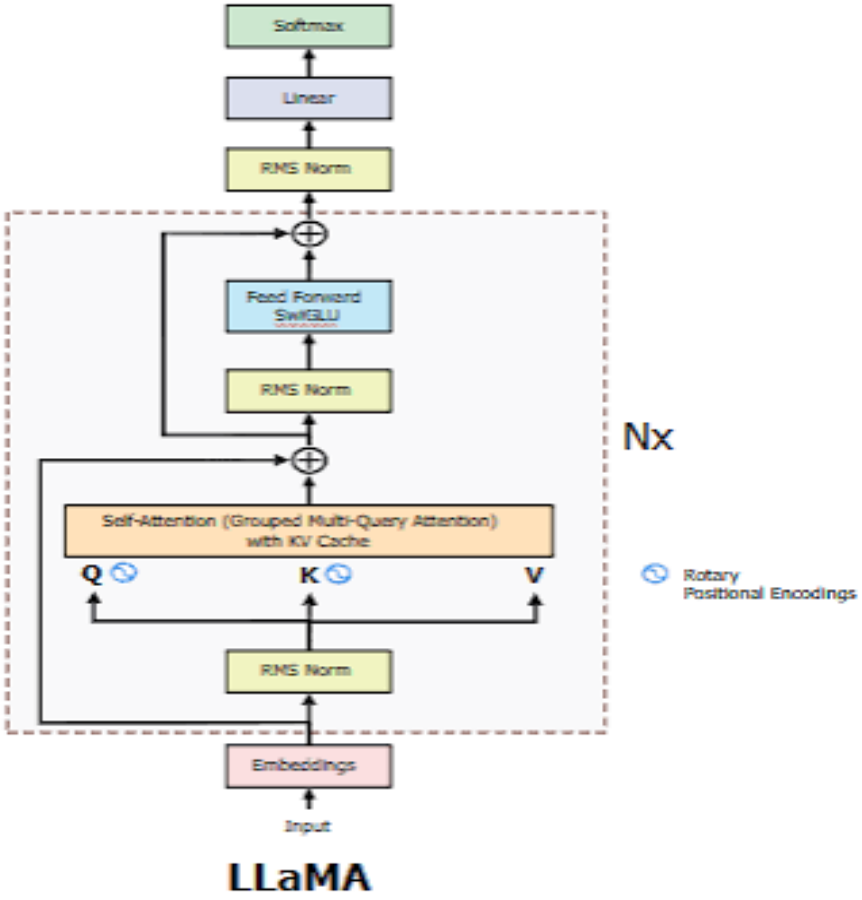
The process continues **token by token** until the response is completed. The generated tokens are converted back into readable text and presented to the user.

Constitutional AI

Claude's development also incorporates **Constitutional AI**, an Anthropic approach that uses a set of principles to guide training and model behavior. It includes processes such as **critique, revision, and preference-based training** to improve alignment.

3.5. LLaMA (Large Language Model Meta AI)

LLaMA (Large Language Model Meta AI) is a family of large language models developed by **Meta AI**. It is based on a **decoder-only Transformer architecture** designed for efficient training and high-quality language generation. LLaMA incorporates improvements such as **RMSNorm, Rotary Positional Embeddings (RoPE), SwiGLU activation, and causal self-attention**.



The LLaMA architecture consists of the following major components:

1. **Tokenization :** The input text is divided into smaller units called **tokens**. Each token is converted into a numerical token ID.
2. **Token Embedding :** Token IDs are converted into dense numerical vectors called **token embeddings**, which represent the semantic information of the tokens.
3. **Transformer Decoder Blocks**

   LLaMA uses multiple identical **Transformer decoder blocks**. Each block contains:

   - **RMSNorm**
   - **Causal Self-Attention**

- **RoPE**
- **Residual Connection**
- **RMSNorm**
- **SwiGLU Feed-Forward Network**
- **Residual Connection**

4. RMSNorm

LLaMA uses **Root Mean Square Normalization (RMSNorm)** instead of conventional Layer Normalization. It improves training stability and reduces computational overhead.

5. Causal Self-Attention

The model uses **causal self-attention**, where each token can attend only to itself and previously generated tokens. This prevents the model from accessing future tokens during generation.

6. Rotary Positional Embedding (RoPE)

RoPE provides positional information by rotating the Query and Key representations according to \ their token positions. It helps the model understand the relative positions of tokens.

7. SwiGLU Feed-Forward Network

LLaMA uses a **SwiGLU-based FFN** instead of the traditional ReLU-based feed-forward network. It improves the model's ability to learn complex relationships in the data.

8. Residual Connections

Residual connections add the input of a sublayer to its output:

Output=Input+Sublayer(Input)Output = Input + Sublayer(Input)

They help preserve information and improve gradient flow during training.

9. Final RMSNorm and Output Layer

After passing through all Transformer blocks, the representation is normalized using **final RMSNorm**. A linear projection then converts it into **logits over the vocabulary**.

10. Autoregressive Generation

The logits are converted into probabilities, and the model selects the next token. The newly generated token is added to the sequence, and the process continues until the response is complete.

Example

For the input:

“The student is learning”

LLaMA processes the tokens and predicts the next token, such as:

“AI”

The sequence becomes:

“The student is learning AI”

The model then predicts the following token. This continues **one token at a time**.

3.6. Text Generation Pipelines and APIs (OpenAI, HuggingFace)

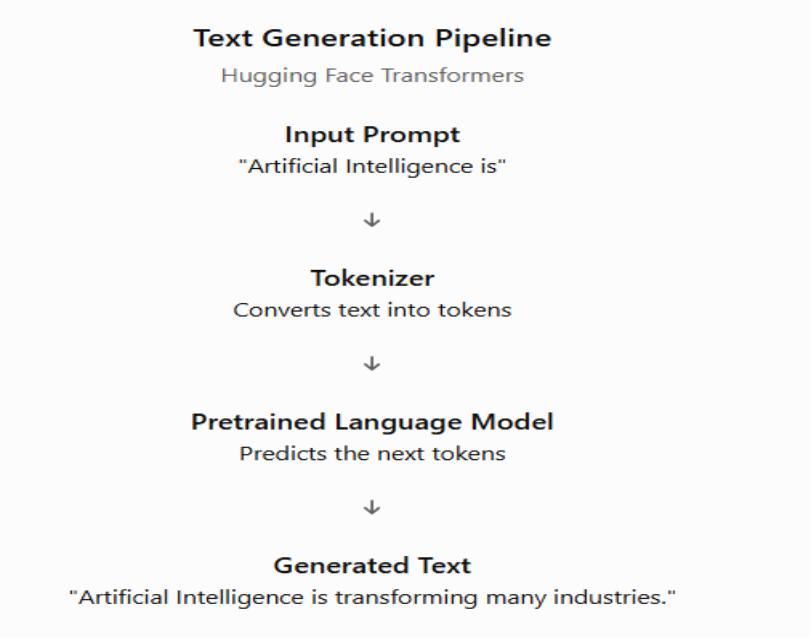
Text generation is a fundamental application of Generative AI in which a language model produces human-like text based on a given prompt or input.

There are two common approaches to implementing text generation:

1. Text Generation Pipeline: A high-level interface that simplifies the process of loading a pretrained model and generating text.
2. Text Generation API: A programmatic interface that allows applications to send prompts to a language model and receive generated text through API requests.

1. Text Generation Pipeline

A pipeline is a high-level abstraction provided by libraries such as Hugging Face Transformers. It hides complex operations such as tokenization, model execution, and decoding.



Python program using Hugging Face

```
Install the required libraries:  pip install transformers torch
from transformers import pipeline
# Create a text generation pipeline
generator = pipeline("text-generation",model="distilgpt2")
# Generate text
result = generator( "Artificial Intelligence
                    is",max_new_tokens=30,do_sample=True,
                    temperature=0.7)
print(result[0]["generated_text"])
```

#Output

Loading weights: 100% 76/76 [00:00<00:00, 2106.70it/s]
[transformers] Both `max_new_tokens` (=50) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Artificial Intelligence is rapidly evolving and the ability to detect patterns of cognitive decline in humans is at a critical stage, and it is now the most important question of all.

Explanation:

- pipeline("text-generation") creates a text generation pipeline.
- distilgpt2 is a pretrained language model.
- max_new_tokens=30 limits the number of newly generated tokens.
- do_sample=True enables sampling for varied outputs.
- temperature=0.7 controls the randomness of token selection.

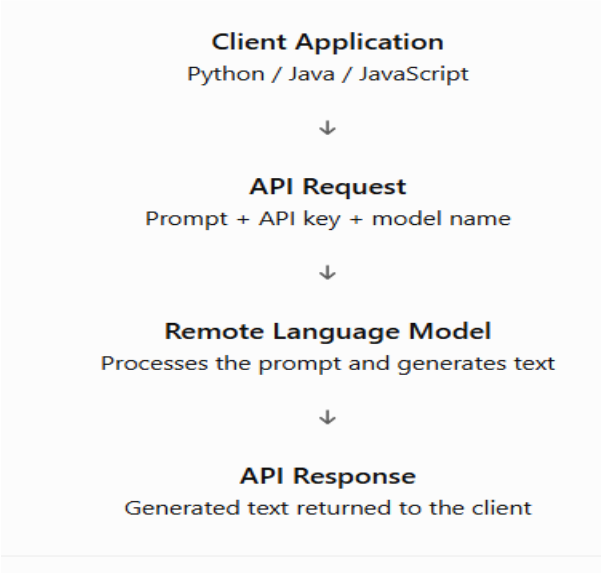
The model generates text locally after downloading its model files. No API key is required for this example.

2. Text Generation API

An API (Application Programming Interface) allows a client application to communicate with a remote language model hosted by a service provider.
Examples include:

- OpenAI API — Text generation using OpenAI models.
- Hugging Face Inference Providers — Access to hosted models through APIs.

API Workflow



OpenAI Text Generation API

The OpenAI API provides access to hosted language models. The API key authenticates the request, while the model generates a response based on the input prompt.

Python program

```
Install the OpenAI library: pip install openai
Set your API key as an environment variable:
$env:OPENAI_API_KEY="your_api_key"

from openai import OpenAI
client = OpenAI()
response = client.responses.create(
                              model="gpt-4.1-mini",
                              input="Explain Artificial Intelligence in two
                              sentences."
)
print(response.output_text)
```

Explanation

- OpenAI() creates the API client.
- The API key is read from the OPENAI_API_KEY environment variable.
- client.responses.create() sends the prompt to the model.
- response.output_text retrieves the generated text.

Use a current model identifier supported by your OpenAI account. See the official OpenAI Responses API documentation for the current API details.

Hugging Face Text Generation API

Hugging Face provides hosted inference services that allow applications to generate text without downloading and running the model locally.

Python program

```
Install the required library: pip install huggingface_hub
# Windows PowerShell
$env:HF_TOKEN="your_huggingface_token"
import os
from huggingface_hub import InferenceClient
client = InferenceClient(
                          api_key=os.environ["HF_TOKEN"]
)
response = client.text_generation(
                    prompt="Artificial Intelligence is",
                    model="HuggingFaceTB/SmolLM2-1.7B-Instruct",
                    max_new_tokens=30
)
```

```
print(response)
```

The model must be available through the selected inference provider. The API token is used to authenticate the request.

Pipeline vs API

Feature	Text Generation Pipeline	Text Generation API
Execution	Usually local	Usually remote
Model	Downloaded pretrained model	Hosted model
API key	Not normally required	Required by the provider
Internet	Needed for initial model download	Needed for API requests
Main library	Transformers	OpenAI SDK / Hugging Face Hub
Use case	Learning, experimentation, local applications	Production applications and hosted inference

3.7. Prompt Engineering with GPT Models

Prompt Engineering is the process of designing, structuring, and refining prompts to guide **GPT models** to generate accurate, relevant, and desired outputs. A well-designed prompt improves the quality, consistency, and usefulness of the model's response.

Main Components of a Prompt

1. **Role** – Defines the role of the GPT model.
2. **Task/Instruction** – Clearly specifies what the model must do.
3. **Context** – Provides necessary background information.
4. **Constraints** – Specifies limitations such as length, style, or content.
5. **Examples** – Shows the expected input-output pattern.
6. **Output Format** – Defines the required response structure.

Important Prompt Engineering Techniques

1. **Zero-Shot Prompting**
   The model performs a task without any examples.
   Example: "Translate this sentence into Telugu."
2. **Few-Shot Prompting**
   A few examples are provided to guide the model.
   Example:
   Good → మంచిది

Welcome → స్వాగతం

Thank you → ?

3. **Role Prompting**
   Assigns a specific role to the model.
   Example: "Act as a Java programming instructor and explain inheritance."
4. **Contextual Prompting**
   Provides relevant background information before the task.
   Example: "Explain Transformers to students who already know neural networks."
5. **Output-Control Prompting**
   Specifies the desired format of the response.
   Example: "Explain GPT architecture using headings, bullet points, and an example."
6. **Prompt Refinement**
   The prompt is repeatedly modified to improve the quality of the generated response.

Example

Basic Prompt:

Explain Machine Learning.

Well-Engineered Prompt:

Act as a Computer Science instructor. Explain Machine Learning to B.Tech students in simple language. Cover its definition, types, applications, and one practical example. Use numbered headings.

The second prompt provides **role, audience, task, scope, and output structure**, giving better control over the GPT response.

Advantages

- Improves response relevance and accuracy
- Controls output format and style
- Reduces ambiguity
- Helps perform specialized tasks
- Enables consistent results
- Makes GPT models more effective for education, coding, research, and content generation.

3.8. Fine-tuning vs Instruct Tuning

Fine-Tuning

Fine-tuning further trains a pre-trained model using a **task-specific dataset** so that the model learns a particular task, domain, style, or behavior.

Example: Sentiment Analysis

Suppose a pre-trained language model is fine-tuned using thousands of movie reviews:

Training Data:

- "The movie was excellent." → **Positive**
- "The movie was boring." → **Negative**
- "The acting was average." → **Neutral**

After fine-tuning:

Input: "The movie was fantastic and enjoyable."

Output: Positive

Here, the model has been specifically trained to perform **sentiment classification**.

Simple flow:

Pre-trained Model → Task-Specific Dataset → Fine-Tuning → Specialized Model

Instruct-Tuning

Instruct-tuning trains a pre-trained model using **instruction–response pairs** so that it learns to understand and follow natural-language instructions.

Example: Translation

Instruction: Translate the following sentence into Telugu.

Input: "Artificial Intelligence is transforming education."

Response: "కృత్రిమ మేధస్సు విద్యను మారుస్తోంది."

The model learns to **follow the instruction** and generate the required response.

Simple flow:

Pre-trained Model → Instruction–Response Dataset → Instruct-Tuning → Instruction-Following Model

Major Difference

Fine-Tuning:

"Learn to perform a specific task."

Instruct-Tuning:

"Learn to follow different natural-language instructions."

Example:

- Fine-tuning → **Movie Review → Positive/Negative/Neutral**
- Instruct-tuning → **"Summarize this movie review." → Summary**

3.9. Retrieval-Augmented Generation (RAG)

RAG addresses one of the core training challenges from hallucination by grounding the model's answer in real, retrieved documents rather than relying purely on parametric memory learned during pretraining.

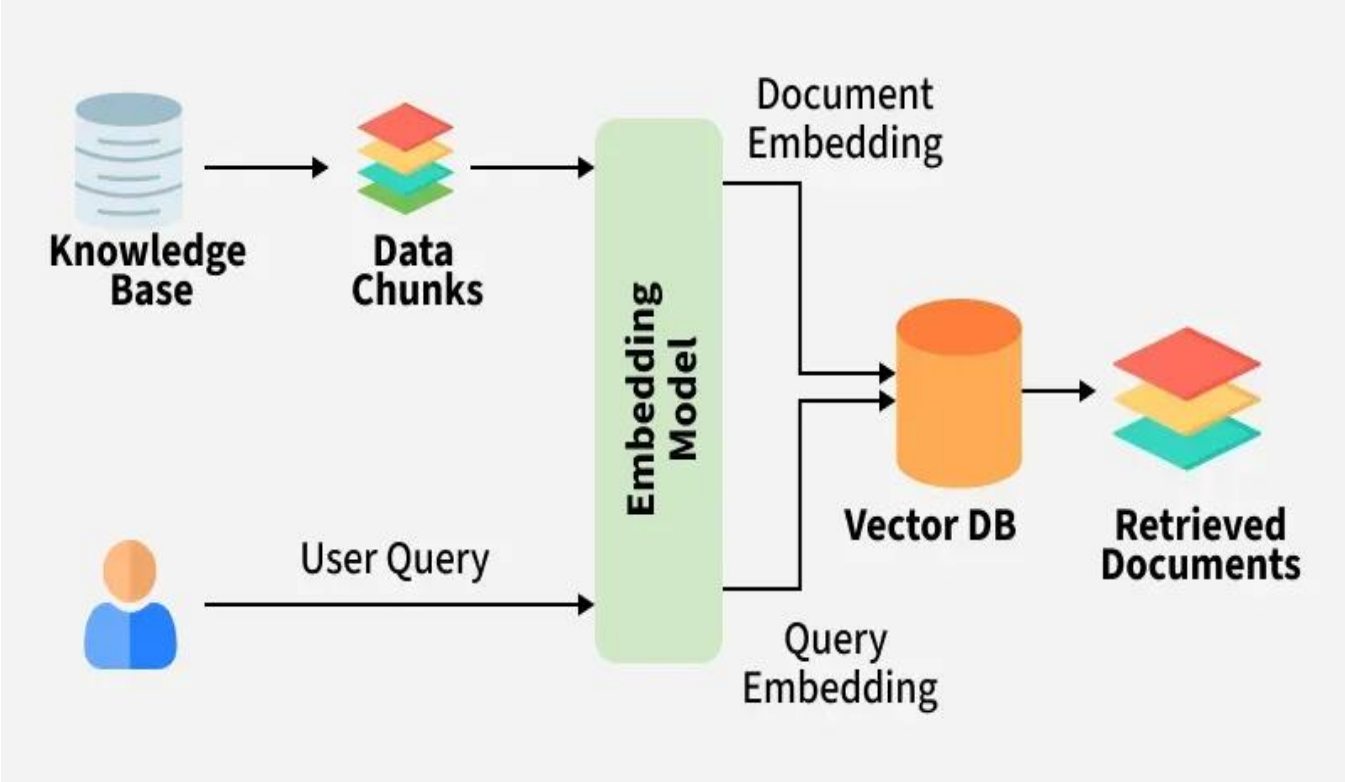
> **Core Definition**
>
> *Retrieval-Augmented Generation combines a retrieval system (typically a vector database of document embeddings) with a generative LLM: relevant documents are retrieved based on the user's query and injected into the prompt as context, so the model generates its answer grounded in that retrieved evidence.*

Retrieval-Augmented Generation (RAG) is a Generative AI technique that combines **information retrieval** with **text generation**.

RAG = Retrieval + Context Augmentation + Generation

The central idea is to **retrieve relevant knowledge first and then allow the LLM to generate an answer based on that knowledge**.

Instead of depending only on the knowledge stored in a pre-trained Large Language Model (LLM), RAG first **retrieves relevant information from an external knowledge source** and then provides that information to the LLM to generate a more accurate and context-aware response.



Basic idea:

User Query → Retrieve Relevant Information → Add Context → LLM → Generated Answer

Components of RAG

The main components of RAG are:

a) **External Knowledge Source:** Stores domain specific or general information like documents, APIs or databases.
b) **Text Chunking and Preprocessing:** Breaks large text into smaller, manageable chunks and cleans it for consistency.
c) **Embedding Model:** Converts text into numerical vectors that capture semantic meaning.
d) **Vector Database:** Stores embeddings and enables similarity search for fast information retrieval.
e) **Query Encoder:** Transforms the user's query into a vector for comparison with stored embeddings.
f) **Retriever:** Finds and returns the most relevant chunks from the database based on query similarity.
g) **Prompt Augmentation Layer:** Combines retrieved chunks with the user's query to provide context to the LLM.
h) **LLM (Generator):** Generates a grounded response using both the query and retrieved knowledge.
i) **Updater (Optional):** Regularly refreshes and re-embeds data to keep the knowledge base up to date.

RAG Process – Step by Step

The system first searches external sources for relevant information based on the user's query instead of relying only on existing training data.

Step 1: Document Collection

Relevant documents such as PDFs, webpages, textbooks, reports, or databases are collected.

Step 2: Chunking

Large documents are divided into smaller meaningful sections called **chunks**.

Step 3: Embedding

Each chunk is converted into a numerical vector using an embedding model.

Step 4: Storage

The vectors are stored in a **vector database** such as ChromaDB, FAISS, or Pinecone.

Step 5: Query Processing

The user's question is converted into an embedding.

Step 6: Retrieval

The retriever compares the query embedding with stored embeddings and retrieves the most relevant chunks.

Step 7: Augmentation

The retrieved chunks are combined with the original query to create a context-rich prompt.

Step 8: Generation

The LLM uses this context to generate the final answer.

Example

Suppose a university has a document containing:

"Students must complete a minimum of 8 weeks of internship and submit the internship report to their faculty mentor."

User asks:

"How long should the internship be?"

RAG retrieves the relevant section from the university document.

The LLM receives:

Question: How long should the internship be?

Retrieved Context: Students must complete a minimum of 8 weeks of internship.

Generated Answer:

"The internship should be completed for a minimum of 8 weeks."

Here, the LLM generates the answer using **retrieved organizational information** rather than relying only on its pre-trained knowledge.

What Problems does RAG solve

1. **Hallucinations**: Traditional generative models can produce incorrect information. RAG reduces this risk by retrieving verified, external data to ground responses in factual knowledge.
2. **Outdated Information**: Static models rely on training data that may become outdated. It dynamically retrieves latest information ensuring relevance and accuracy in real time**.**
3. **Contextual Relevance**: Generative models often struggle with maintaining context in complex or multi turn conversations. RAG retrieves relevant documents to enrich the context improving coherence and relevance.
4. **Domain Specific Knowledge**: Generic models may lack expertise in specialized fields. It integrates domain specific external knowledge for tailored and precise responses.
5. **Cost and Efficiency**: Fine tuning large models for specific tasks is expensive. It eliminates the need for retraining by dynamically retrieving relevant data reducing costs and computational load.
6. **Scalability Across Domains**: It is adaptable to diverse industries from healthcare to finance without extensive retraining making it highly scalable.

Challenges

1. **Complexity**: Combining retrieval and generation adds complexity to the model requires careful tuning and optimization to ensure both components work seamlessly together.

2. **Latency**: The retrieval step can introduce latency making it challenging to deploy RAG models in real time applications.
3. **Quality of Retrieval**: The overall performance heavily depends on the quality of the retrieved documents. Poor retrieval can lead to suboptimal generation, undermining the model's effectiveness.
4. **Bias and Fairness**: It can inherit biases present in the training data or retrieved documents, necessitating ongoing efforts to ensure fairness and mitigate biases.

RAG Applications

1. **Question-Answering Systems**: It enables chatbots or virtual assistants to pull information from a knowledge base or documents and generate accurate, context aware answers.
2. **Content Creation and Summarization:** It can gather information from multiple sources and generate concise, simplified summaries or articles.
3. **Conversational Agents and Chatbots:** It enhances chatbots by grounding their responses in reliable data making interactions more informative and personalized.
4. **Information Retrieval:** Goes beyond traditional search by retrieving documents and generating meaningful summaries of their content.
5. **Educational Tools and Resources:** Provides students with explanations, diagrams or multimedia references tailored to their queries.

3.10. Evaluation Metrics — BLEU, ROUGE, Perplexity

Evaluation metrics are used to measure the **quality, accuracy, and fluency of text generated by Generative AI models**. Three commonly used metrics are **BLEU, ROUGE, and Perplexity**.

1. BLEU — Bilingual Evaluation Understudy

BLEU measures the **similarity between generated text and a reference text** using matching **n-grams**.

- Mainly used for **machine translation**.
- Compares generated output with one or more reference answers.
- Uses **precision** of matching words or word sequences.
- BLEU score generally ranges from **0 to 1** or **0 to 100**.
- **Higher BLEU → greater similarity to the reference text.**

Example:

Reference: *The student is learning Python programming.*

Generated: *The student is learning Python.*

The generated sentence shares several word sequences with the reference, resulting in a relatively higher BLEU score.

2. ROUGE — Recall-Oriented Understudy for Gisting Evaluation

ROUGE measures the **overlap between generated text and reference text**, with emphasis on **recall**.

It is widely used for **text summarization**.

Common variants:

- **ROUGE-1:** Unigram/word overlap
- **ROUGE-2:** Bigram overlap
- **ROUGE-L:** Longest Common Subsequence (LCS)

Example:

Reference: *Artificial Intelligence helps computers perform intelligent tasks.*

Generated: *AI helps computers perform intelligent tasks.*

ROUGE evaluates how much of the important content from the reference is captured by the generated text.

Higher ROUGE → greater content overlap with the reference.

3. Perplexity (PPL)

Perplexity measures how well a language model **predicts the next token in a sequence**.

It indicates the model's **uncertainty** while generating text.

$$PPL = e^{-\frac{1}{N}\sum_{i=1}^{N} \quad logP(w_i)}$$

Where:

- N = number of tokens

- $P(w_i)$ = probability assigned to the actual token

Interpretation:

- **Lower Perplexity → better prediction and greater confidence**
- **Higher Perplexity → greater uncertainty**

Example:

If a model predicts:

The student is learning ___.

with high probability for **Python**, its perplexity for the sequence will generally be lower than if it assigns similar low probabilities to many possible words.

Metric	What It Measures	Formula Basis	Typical Use	Key Limitation
BLEU	N-gram precision overlap between generated and reference text	Precision of matching n-grams, with a brevity penalty	Machine translation quality	Penalises valid paraphrasing; ignores meaning
ROUGE	N-gram / longest-common-subsequence recall overlap	Recall-oriented overlap (ROUGE-N, ROUGE-L)	Summarisation quality	Favours longer outputs; weak on semantic correctness
Perplexity	How well a language model predicts held-out text	2^(cross-entropy loss) — lower is better	Comparing language model quality during training	Doesn't measure factual accuracy or usefulness

Analogy

BLEU and ROUGE compare generated text with reference text, whereas Perplexity evaluates the language model's predictive uncertainty.

3.11. Building LLM-based Apps with LangChain

LangChain is an open-source framework for developing applications powered by **Large Language Models (LLMs)**. Instead of interacting with an LLM through a single prompt, LangChain provides components for building complete applications that can:

- Generate and understand text
- Maintain conversation history
- Connect LLMs with external data
- Search documents
- Perform Retrieval-Augmented Generation (RAG)
- Use external tools and APIs
- Execute multi-step workflows
- Build intelligent agents
- Connect with vector databases
- Create domain-specific AI assistants

The basic idea is:

LLM + Prompt + Data + Tools + Memory + Workflow = LLM Application

A direct LLM API call is useful for simple text generation:

User → Prompt → LLM → Response

However, real-world applications are more complex.

For example, a **College Knowledge Assistant** may need to:

1. Accept a student's question.
2. Search university documents.
3. Retrieve relevant information.
4. Send the information to an LLM.
5. Generate an answer.
6. Maintain conversation context.
7. Cite the source documents.

LangChain helps organize these operations into a structured application.

Major Components of LangChain

A LangChain application can be understood through several major building blocks.

Component	Brief Description
LLMs / Chat Models	Provide the core language-generation capability for understanding prompts and producing responses.
Prompt Templates	Define reusable prompts with variables that can be dynamically filled with user input or retrieved information.

Output Parsers	Convert LLM responses into a required format such as text, JSON, lists, or structured objects.
Chains / Runnables	Connect multiple processing steps to create an end-to-end workflow, such as Prompt → Model → Parser.
Document Loaders	Load data from sources such as PDFs, text files, web pages, CSV files, databases, and other documents.
Text Splitters	Divide large documents into smaller chunks so they can be efficiently processed and retrieved.
Embeddings	Convert text into numerical vectors that represent its semantic meaning.
Vector Stores	Store embeddings and enable similarity-based searching of relevant information.
Retrievers	Find and return the most relevant documents or chunks for a given user query.
Tools	Allow an LLM application to interact with external functions, APIs, databases, calculators, search systems, etc.
Agents	Enable an LLM to dynamically select and use appropriate tools to complete a task.
Message History / State	Maintains conversation messages and application state so that interactions can retain context.
LangGraph	Supports complex, stateful, multi-step workflows and agent-based applications with controlled execution.

LLM and Chat Models

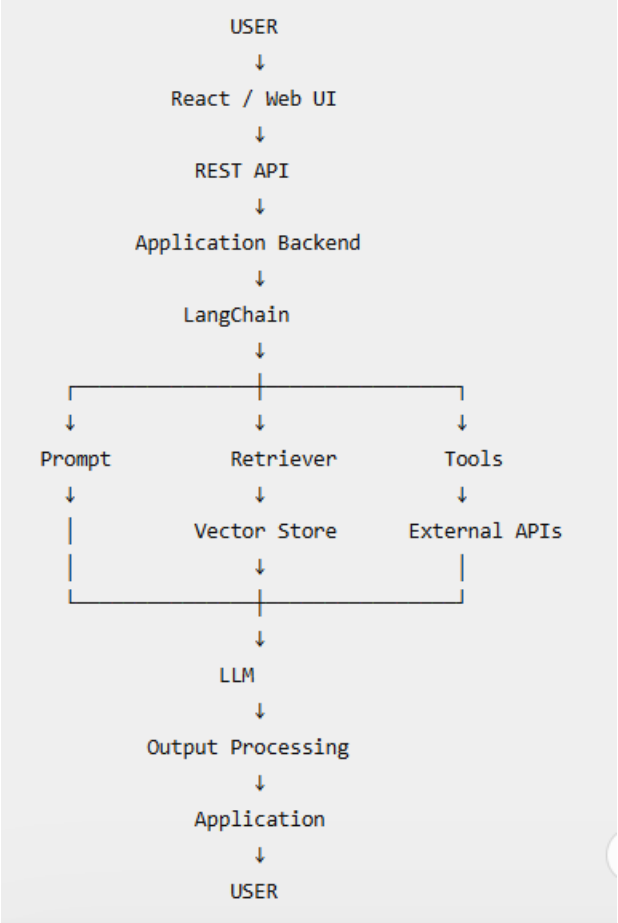
The **LLM or chat model** is the central intelligence of the application.

LangChain can provide a common programming interface for working with different model providers.

Examples include:

- OpenAI models
- Anthropic models
- Google models
- Hugging Face models
- Ollama/local models
- Other compatible providers

Complete LangChain Application Architecture



LangChain Application Development Workflow

Developing an LLM-based application with LangChain can be organized into the following step-by-step workflow.

1. **Define the Application Problem** – Identify the purpose, users, inputs, and expected outputs of the application.
2. **Select the LLM / Chat Model** – Choose a suitable language model based on accuracy, cost, speed, context length, and application requirements.
3. **Set Up the LangChain Environment** – Install LangChain and the required model, database, and integration packages.
4. **Configure API Credentials** – Securely configure API keys or authentication credentials required to access the selected LLM and other services.
5. **Design Prompt Templates** – Create reusable prompts with variables for user queries, instructions, and contextual information.
6. **Connect Prompt Templates with the LLM** – Create the basic LangChain pipeline that sends formatted prompts to the language model.

7. **Add Output Parsers** – Convert the LLM response into the required format, such as text, JSON, lists, or structured objects.
8. **Collect Required Data** – Identify and gather the documents or external data required by the application.
9. **Load Documents** – Use LangChain document loaders to import PDFs, text files, web pages, CSV files, databases, and other sources.
10. **Split Documents into Chunks** – Divide large documents into smaller, meaningful sections for efficient processing and retrieval.
11. **Generate Embeddings** – Convert document chunks into numerical vector representations that capture their semantic meaning.
12. **Store Embeddings in a Vector Store** – Store the generated vectors in a vector database for efficient similarity search.
13. **Create a Retriever** – Retrieve the most relevant document chunks based on the user's query.
14. **Build the RAG Pipeline** – Combine the user's question, retrieved context, prompt, and LLM to generate a grounded response.
15. **Add Conversation History / State** – Maintain relevant previous interactions so the application can support contextual conversations.
16. **Integrate External Tools and APIs** – Connect calculators, databases, search systems, REST APIs, and other external functions.
17. **Build Agents for Dynamic Tasks** – Enable the LLM to select and use appropriate tools dynamically to accomplish complex tasks.
18. **Develop the Application Interface** – Connect the LangChain backend with a web, mobile, chatbot, or other user interface.
19. **Test and Evaluate the Application** – Evaluate accuracy, retrieval quality, hallucinations, latency, cost, reliability, and security.
20. **Deploy and Monitor the Application** – Deploy the application to a production environment and continuously monitor its performance, usage, and failures.

Advantages of LangChain

1. Modularity: Individual components can be replaced independently.
2. Model flexibility: Applications can work with different model providers.
3. RAG support: Useful abstractions exist for document ingestion and retrieval.
4. Tool integration: LLMs can interact with external systems.
5. Reusable prompts: Prompt templates can be reused across applications.
6. Rapid development: Developers can construct prototypes quickly.
7. Complex workflows: Multiple operations can be connected into pipelines.
8. Ecosystem: It integrates with models, vector stores, retrievers, tools, and other AI infrastructure.

Limitations and Challenges

LangChain does not automatically solve every GenAI problem.

Important challenges include:

- LLM hallucinations
- Incorrect retrieved information
- Poor document chunking
- Embedding quality
- Prompt design
- Token consumption
- Latency
- Cost
- Security
- Privacy
- Tool misuse
- Agent unpredictability
- Evaluation difficulty
- Version/API changes in the ecosystem

Important RAG evaluation :

Evaluation Category	Metric	Explanation
Retrieval Evaluation	**Precision**	Measures how many of the retrieved documents or chunks are actually relevant to the user's query.
	Recall	Measures how many of the relevant documents or chunks available in the knowledge base were successfully retrieved.
	Relevance	Measures how closely the retrieved information matches the user's query or information need.
Generation Evaluation	**Faithfulness**	Measures whether the generated answer is supported by the retrieved context and does not introduce unsupported information.
	Answer Relevance	Measures how directly and appropriately the generated answer addresses the user's question.
	Factual Correctness	Measures whether the information in the generated answer is factually accurate.
	Completeness	Measures whether the answer covers the important information required to answer the question adequately.
System Evaluation	**Latency**	Measures the time taken by the application to produce a response.

	Token Usage	Measures the number of input and output tokens processed by the LLM.
	Cost	Measures the computational or API expense required to process requests.
	Reliability	Measures how consistently the application produces successful and stable responses.
	Failure Rate	Measures the percentage of requests that result in errors, unsuccessful retrievals, or unusable responses.

The basic LLM Applications with LangChain demonstrated in Appendix-A

3.12. Assignment Questions

Part A: Two-Mark Questions

1. Define the Transformer architecture and state the role of self-attention.
2. Differentiate between BERT and GPT based on their architectural approach.
3. What are the major architectural differences between GPT-3/4 and encoder-based Transformer models?
4. Define PaLM, Claude, and LLaMA as large language models.
5. What is a text generation pipeline? Mention any two of its components.
6. Differentiate between the OpenAI API and Hugging Face inference APIs.
7. Define prompt engineering and state its importance in LLM applications.
8. What is a few-shot prompt? Give one example of its use.
9. Define fine-tuning and instruction tuning.
10. State any two differences between fine-tuning and instruction tuning.
11. Define Retrieval-Augmented Generation (RAG).
12. What is the role of a vector database in a RAG pipeline?
13. Define BLEU and mention one application of this metric.
14. What is ROUGE? How is it used to evaluate text generation?
15. Define perplexity in the context of language models.
16. Differentiate between BLEU and ROUGE.
17. What is LangChain? State any two components used in LLM applications.
18. What is a prompt template in LangChain?
19. Why is document chunking important in a RAG-based application?
20. Identify two limitations of using an LLM API without retrieval or grounding.

Part B: Ten-Mark Questions

1. **Explain the Transformer architecture with a neat diagram. Discuss self-attention, multi-head attention, positional encoding, feed-forward networks, residual connections, and layer normalization.**
2. **Discuss BERT and GPT Architectures with neat sketch.Compare BERT and GPT architectures in terms of attention mechanisms, training objectives, contextual representation, and practical applications.**
3. **Analyze the architectural approaches and capabilities of GPT-3/4, PaLM, Claude, and LLaMA. Discuss their training approaches, deployment considerations, and application suitability.**
4. Design an end-to-end text generation pipeline using a pretrained LLM. Explain input preprocessing, prompt construction, model inference, decoding strategies, output postprocessing, and error handling.
5. **Develop a Python application using an OpenAI API or a Hugging Face inference interface to generate text. Explain API configuration, request construction, response processing, parameter selection, and exception handling.**
6. Analyze zero-shot, one-shot, few-shot, and chain-of-thought prompting approaches. Design suitable prompts for a customer-support application and explain how their outputs can be assessed.
7. Design an effective prompt engineering strategy for an LLM-based educational assistant. Include role definition, context, task instructions, constraints, output formatting, examples, and validation criteria.
8. **Compare traditional fine-tuning and instruction tuning in terms of objectives, datasets, training procedures, computational requirements, and use cases. Recommend a suitable approach for a domain-specific chatbot with justification.**
9. Design a fine-tuning or instruction-tuning workflow for a customer-service LLM. Explain dataset preparation, data quality checks, training, validation, overfitting prevention, and deployment.
10. **Explain the architecture and working of a Retrieval-Augmented Generation system with a neat diagram. Discuss document ingestion, chunking, embeddings, vector indexing, retrieval, context construction, and answer generation.**
11. **Design a RAG-based question-answering system for a university knowledge repository. Explain document processing, embedding generation, similarity search, prompt construction, source citation, and response generation.**
12. Analyze the impact of chunk size, chunk overlap, embedding models, top-\(k\) retrieval, and similarity thresholds on the quality of RAG-based responses.
13. **Compare BLEU, ROUGE, and perplexity based on their objectives, calculation principles, strengths, limitations, and appropriate use cases.**
14. Evaluate a machine-generated summary using ROUGE and a machine-translated sentence using

BLEU. Explain how reference texts affect metric scores and why automatic metrics may not fully capture semantic quality.

15. Analyze how perplexity reflects a language model's predictive performance. Explain the relationship between token probabilities, cross-entropy, and perplexity, and discuss why perplexity alone cannot establish response quality.
16. **Design a comprehensive evaluation framework for an LLM application using BLEU, ROUGE, perplexity, factual accuracy, hallucination rate, and human evaluation. Justify the selection of metrics for different tasks.**

17. **Explain the major components of LangChain, including models, prompt templates, output parsers, retrievers, chains, and tools. Illustrate how they work together in an LLM application.**
18. Develop the design for a LangChain-based conversational question-answering application. Explain model integration, prompt templates, conversation history, retrieval, response generation, and error handling.
19. **Design an end-to-end LangChain application that uses RAG to answer questions from PDF documents. Describe document loading, text splitting, embeddings, vector storage, retrieval, prompt construction, and answer generation.**
20. Evaluate the suitability of a direct LLM API, a fine-tuned model, and a LangChain-based RAG application for a university academic assistant. Compare implementation complexity, cost, maintainability, factual grounding, and update frequency, then recommend an architecture.

Appendix-A
Basic LLM Applications with LangChain

Problem Statement: 1. *AI Chatbot*

Develop a chatbot using LangChain that interacts with users and provides answers to their questions using an LLM.

Explanation:

1. AI Chatbot is an application that interacts with users and answers their questions using an LLM.
2. LangChain connects the user's input with the LLM and processes the request.
3. The LLM generates a suitable response, which is displayed to the user through the chatbot.

Program:

```
!pip install -q -U langchain langchain-google-genai

import os

from getpass import getpass

from langchain_google_genai import ChatGoogleGenerativeAI

print("Enter your Gemini API Key:")

os.environ["GOOGLE_API_KEY"] = getpass()

llm = ChatGoogleGenerativeAI(

   model="gemini-3.8-flash",

   temperature=0.7

)print("\n==================================")

print("         AI CHATBOT")

print("==================================")

print("Type 'exit' to stop the chatbot.\n")

while True:

 user_input = input("You: ")

  if user_input.lower() == "exit":

       print("Chatbot: Goodbye!")

       break

  try:
```

```
        response = llm.invoke(user_input)
        if isinstance(response.content, list):
            answer = ""
             for item in response.content:
               if isinstance(item, dict) and item.get("type") == "text":
                   answer += item.get("text", "")
             else:
                answer = response.content
        print("Chatbot:", answer)
        print() except Exception as e:
  if "429" in str(e) or "RESOURCE_EXHAUSTED" in str(e):
            print("\nChatbot: Gemini API quota has been exhausted.")
            print("Please wait for the quota to reset or use a project")
            print("with an available Gemini API quota.\n")
            break
  else:  print("\nError:", e)
            print()
```

Output:

```
Enter your Gemini API Key:
.........
==================================
        AI CHATBOT
==================================
Type 'exit' to stop the chatbot.
You: hi
Chatbot: Hello! How can I help you today?
You: exit
Chatbot: Goodbye!
```

Problem Statement: 2. *Text Summarizer*

Develop an application using LangChain that summarizes long text into short and meaningful content.

Explanation:1. Text Summarizer is an application that converts long text into short and meaningful content.

2.LangChain connects the input text with an LLM, which identifies and extracts the important information.

3.The application displays the important points as a clear and concise summary.

Program:

```
!pip install -q -U langchain langchain-google-genai

import os

import time

from getpass import getpass

from langchain_google_genai import ChatGoogleGenerativeAI

print("Enter your Gemini API Key:")

os.environ["GOOGLE_API_KEY"] = getpass()

llm = ChatGoogleGenerativeAI(

   model="gemini-3.8-flash",

   temperature=0.3

)

print("\n==================================")

print("       AI TEXT SUMMARIZER")

print("==================================")

text = input("\nEnter the text to summarize:\n")

prompt = f"""

Summarize the following text into a short and meaningful summary.

Keep only the important points.

Text:

{text}
```

```
"""
for attempt in range(3):
    try:
        response = llm.invoke(prompt)
        if isinstance(response.content, list):
            summary = ""
            for item in response.content:
                if isinstance(item, dict) and item.get("type") == "text":
                    summary += item.get("text", "")
        else:
            summary = response.content
        print("\n==================================")
        print("            SUMMARY")
        print("==================================")
        print(summary)
        break
    except Exception as e:
        error_message = str(e)
        if "503" in error_message or "UNAVAILABLE" in error_message:
            print("\nGemini server is currently busy.")
            print("Retrying... Attempt", attempt + 1, "of 3")
            time.sleep(5)
  elif "429" in error_message or "RESOURCE_EXHAUSTED" in error_message:
  print("\nGemini API quota has been exhausted.")
            print("Please wait until the quota resets.")
            break
        else:
print("\nError occurred:")
```

```
        print(e)
        break
```

Output:

```
Enter your Gemini API Key:
.....
      AI TEXT SUMMARIZER
==================================
Enter the text to summarize:
```

Artificial Intelligence is a technology that enables computers and machines to perform tasks that normally require human intelligence. It is used in many areas such as healthcare, education, banking, transportation, and entertainment. Machine Learning is a part of Artificial Intelligence that allows computers to learn from data and improve their performance without being explicitly programmed.

```
        SUMMARY
==================================
**Summary:**
```

Artificial Intelligence (AI) enables machines to perform human-like tasks across various industries. A key subset of AI, Machine Learning, allows systems to automatically learn and improve from data without being explicitly programmed.

Problem Statement: 3. *Question Answering System*

Develop an LLM-based application using LangChain that accepts a question and generates an appropriate answer.

Explanation:

1. Question Answering System accepts a question from the user as input and uses an LLM to process it.
2. LangChain connects the user's question with the LLM and manages the interaction.
3. The LLM generates an appropriate and meaningful answer, which is displayed to the user.

Program:

```
!pip install -q -U langchain langchain-google-genai

import os

from getpass import getpass

from langchain_google_genai import ChatGoogleGenerativeAI

print("Enter your Gemini API Key:")

os.environ["GOOGLE_API_KEY"] = getpass()

llm = ChatGoogleGenerativeAI(

   model="gemini-3.8-flash",

   temperature=0.5)

print("\n==================================")

print("    QUESTION ANSWERING SYSTEM")

print("==================================")

print("Type 'exit' to stop.\n")

while True:

   question = input("Enter your question: ")

   if question.lower() == "exit":

      print("System: Goodbye!")

      break

   prompt = f"""
```

```
Answer the following question clearly and accurately.

Question:

{question}

""" try:

    response = llm.invoke(prompt)

if isinstance(response.content, list):

        answer = ""

        for item in response.content:

            if isinstance(item, dict) and item.get("type") == "text":

                answer += item.get("text", "")

    else:

        answer = response.content

    print("\nAnswer:", answer)

    print()

except Exception as e:

    print("\nError:", e)

    print()
```

Output:

```
Enter your Gemini API Key:

.....

QUESTION ANSWERING SYSTEM

Type 'exit' to stop.

Enter your question: what is java

Answer: **Java** is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible. Developed by James Gosling at Sun Microsystems and released in 1995 (now owned by Oracle), it is one of the most popular and enduring programming languages in the world.

Enter your question: exit

System: Goodbye!
```

Problem Statement: 4. *AI Text Generator*

Develop an application using LangChain that generates text based on a user's given topic or prompt.

Explanation:

1. AI Text Generator accepts a topic or prompt from the user as input.
2. LangChain connects the prompt with an LLM, which understands the given topic.
3. The LLM generates relevant and meaningful text based on the user's prompt.

Program:

```
!pip install -q -U langchain langchain-google-genai

import os

from getpass import getpass

from langchain_google_genai import ChatGoogleGenerativeAI

print("Enter your Gemini API Key:")

os.environ["GOOGLE_API_KEY"] = getpass()

llm = ChatGoogleGenerativeAI(

   model="gemini-3.8-flash",

   temperature=0.7

)

print("\n==================================")

print("        AI TEXT GENERATOR")

print("==================================")

topic = input("\nEnter a topic or prompt: ")

prompt = f"""

Generate meaningful and relevant text based on the following topic.

Topic:

{topic}

"""

Try
```

```
  response = llm.invoke(prompt)
    if isinstance(response.content, list):
        generated_text = ""
        for item in response.content:
            if isinstance(item, dict) and item.get("type") == "text":
                generated_text += item.get("text", "")
else:
        generated_text = response.content
    print("\n==================================")
    print("       GENERATED TEXT")
    print("==================================")
    print(generated_text)
except Exception as e:
    print("\nError:", e)
```

Output:

```
Enter your Gemini API Key:
.....
       AI TEXT GENERATOR
```

```
Enter a topic or prompt: Enter a topic or prompt: Write a short paragraph about Artificial Intelligence
       GENERATED TEXT
==================================
```

Artificial Intelligence (AI) represents a transformative leap in modern technology, enabling machines to learn, reason, and perform tasks that traditionally required human intellect. From powering virtual assistants and medical diagnostic tools to driving autonomous vehicles, AI is rapidly reshaping the way we live, work, and solve complex problems. By analyzing vast amounts of data at unprecedented speeds, these systems unlock new frontiers in efficiency and innovation across every industry. As this technology continues to evolve, balancing its remarkable capabilities with thoughtful ethical standards will be essential to ensuring that AI serves as a force for good in society.

Problem Statement: 5. *AI Translation App*

Develop a LangChain-based application that translates text from one language to another using an LLM.

Explanation:

1. AI Translation App accepts text from the user along with the source language and target language.
2. LangChain connects the input text with an LLM, which understands the text and translates it into the selected target language.
3. The application displays the translated text as the final output**.**

Program:

```
!pip install -q -U langchain-google-genai

import os

from getpass import getpass

from langchain_google_genai import ChatGoogleGenerativeAI

print("Enter your Gemini API Key:")

os.environ["GOOGLE_API_KEY"] = getpass()

llm = ChatGoogleGenerativeAI(

   model="gemini-3.8-flash",

   temperature=0

)

print("\n==================================")

print("      AI TRANSLATION APP")

print("==================================")

text = input("\nEnter text to translate: ")

source_language = input("Enter source language: ")

target_language = input("Enter target language: ")

prompt = f"""

Translate the following text from {source_language} to {target_language}.

Text:
```

```
{text}
Rules:
- Give only the translated text.
- Do not give explanations.
- Keep the original meaning.
"""
try: response = llm.invoke(prompt)
   if isinstance(response.content, list):
      translation = ""
      for item in response.content:
         if isinstance(item, dict):
            if item.get("type") == "text":
               translation += item.get("text", "")
   else:
      translation = response.content
   print("\n==================================")
   print("          TRANSLATION")
  print("==================================")
   print(translation)
```

Output:

```
Enter your Gemini API Key:
.....
======================================================
      AI TRANSLATION APP:Enter text to translate: How are you
Enter source language: english
Enter target language: telugu
         TRANSLATION
మీరు ఎలా ఉన్నారు?
```

Problem Statement: 6. *AI Content Rewriter*

Develop an application using LangChain that rewrites user-provided text into a clearer and more readable form.

Explanation

1. **AI Content Rewriter** accepts user-provided text as input and processes it using LangChain.
2. **LangChain connects the text with an LLM**, which improves the grammar, clarity, and readability while keeping the original meaning.
3. The application **displays the rewritten text** in a clear and easy-to-understand form.

Program:

```
!pip install -q -U langchain

from langchain_core.runnables import RunnableLambda

def rewrite_content(text):

   rewritten = text

replacements = {

      "is a technology. It helps computers do many things that humans can do.":

      "is a technology that enables computers to perform tasks that normally require human intelligence.",

     "AI is used in healthcare, education and many other fields.":

      "AI is widely used in healthcare, education, and many other fields."

   }

 for old, new in replacements.items():

      rewritten = rewritten.replace(old, new)

  return rewritten

rewriter = RunnableLambda(rewrite_content)

print("==================================")

print("      AI CONTENT REWRITER")

print("==================================")

text = input("\nEnter the text you want to rewrite:\n")
```

```
try:
result = rewriter.invoke(text)
    print("\n==================================")
   print("       REWRITTEN CONTENT")
    print("==================================")
    print(result)
except Exception as e:
 print("\nError:")
   print(e)
```

Output:

```
      AI CONTENT REWRITER
==================================
Enter the text you want to rewrite:
Artificial intelligence is a technology. It helps computers do many things that humans can do. AI is used in healthcare, education and many other fields.
==================================
       REWRITTEN CONTENT
==================================
Artificial intelligence is a technology that enables computers to perform tasks that normally require human intelligence. AI is widely used in healthcare, education, and many other fields.
```